Research Project in Computer Systems Engineering

Final Report

Project #43: Reconfigurable Neural Processing Unit (NPU) for Energy-Efficient AI at the Edge

Pratham Chhabra

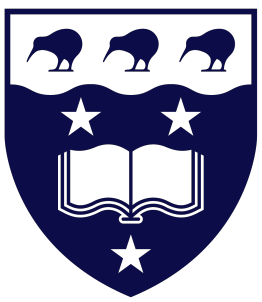
Project Report COMPSYS043-2025

Project Partner: Kristelle Sampang

Supervisor: Dr Morteza Biglari-Abhari

Co-Supervisor: Dr Maryam Hemmati

October 19, 2025



Waipapa Taumata Rau
University of Auckland

COMPSYS043-2025

RECONFIGURABLE NEURAL PROCESSING UNIT (NPU) FOR ENERGY-EFFICIENT AI AT THE EDGE

Pratham Chhabra

ABSTRACT

Presented in this final Research project Report, commissioned as part of the Electrical, Computer, and Software Engineering at The University of Auckland, is the design and implementation of a Reconfigurable Neural Processing Unit for Energy-Efficient Artificial Intelligence at the edge. As the demands for Artificial Intelligence at the edge are increasing, the constraints are becoming tighter for computational cost for many Neural Networks on resource-limited devices. It is established that conventional hardware accelerators consist of fixed architectures that are inefficient when processing sparse data, which is common in optimised neural networks, leading to wasted computational cycles and energy. The proposed solution addresses this problem by creating a hardware-software topology that integrates a pre-processing stage with a dynamically adaptable accelerator. The software component is implemented in Python, and it analyses quantised weight and activation matrices from the AlexNet model, identifying rows and columns with all zeros to generate compacted matrices along with their effective dimensions (M, N, K). The hardware consists of a VHDL-based 8x8 output-stationary systolic array, a reconfigurable control unit that uses these parameters at runtime to adjust the active region, effectively skipping redundant operations. The design was verified using simulations in ModelSim and synthesised for a Cyclone V FPGA using Quartus Prime 18.1 Lite. Simulation results demonstrated a 60.87% average reduction in computational latency per processing tile compared to an unoptimised architecture. Hardware synthesis confirmed these findings, estimating a theoretical throughput of 1.478 GOPs/sec, representing a 2.55x performance speedup over the baseline using the fmax frequency generated by the Quartus timing analyser, 289.28MHz. By combining these findings, the conclusion is made that the co-design approach is effective when provided with a reconfigurable hardware to achieve a practical solution for accelerating sparse CNNs on FPGAs, making running complex AI models more feasible for edge deployment

DECLARATION

Student I hereby declare that:

1. This report is the result of the final year project work carried out by my project partner (see cover page) and I under the guidance of our supervisor (see cover page) in the 2025 academic year at the Department of Electrical, Computer and Software Engineering, Faculty of Engineering, University of Auckland.
2. This report is not the outcome of work done previously.
3. This report is not the outcome of work done in collaboration, except that with a potential project sponsor (if any) as stated in the text.
4. This report is not the same as any report, thesis, conference article or journal paper, or any other publication or unpublished work in any format.

In the case of a continuing project, please state clearly what has been developed during the project and what was available from previous year(s):

Signature: 

Date: 14/10/2025

Table of Contents

Acknowledgements

I would like to express my appreciation towards my supervisors, Dr Morteza Biglari-Abhari and Dr Maryam Hemmati, for their unmatched support for this project throughout the year 2025.

I would also like to express my appreciation to my project partner, Kristelle Sampang, for her contribution and perseverance that she provided alongside me throughout this project.

Glossary of Terms

Term	Definition
Activation Sparsity	A form of sparsity in neural networks caused by activation functions (like ReLU) setting negative outputs to zero. This is dynamic and changes with each input image.
Control Unit	The hardware component responsible for orchestrating data flow, generating timing signals (staggering), and managing the operation of the systolic array.
Edge (AI at the Edge)	The practice of running AI inference directly on local, resource-constrained devices rather than in the cloud.
Fine-Grained Sparsity	See Unstructured Sparsity.
Latency	The time, measured in clock cycles, required to complete a computational task.
Processing Element (PE)	The fundamental building block of a systolic array, typically containing a Multiply-Accumulate (MAC) unit and registers.
Pruning	An optimisation technique that removes redundant or insignificant weights from a neural network to reduce model size and computational complexity.
Quantisation	An optimisation technique that reduces the numerical precision of a model's weights and activations (e.g., from 32-bit float to 8-bit integer) to save memory and simplify hardware.
Sparsity	The property of a matrix containing a large proportion of zero-valued elements.
Staggering	The process of delaying the input of data and weight elements into the systolic array boundaries to ensure they meet at the correct PE at the correct time.
Structured Sparsity	Sparsity that occurs in a regular pattern, such as entire rows or columns of a matrix being zero. This is the type of sparsity targeted by this project's algorithm.
Systolic Array	A parallel computing architecture consisting of a grid of Processing Elements (PEs) designed for high-throughput, pipelined operations like matrix multiplication.
Throughput	A measure of computational performance, indicating the number of operations that can be performed per unit of time (e.g., GOPs/sec).
Tiling	The method of breaking down large matrices into smaller, fixed-size blocks (tiles) to be processed by a hardware accelerator of a fixed size (e.g., 8x8).
Unstructured Sparsity	Sparsity where zero-valued elements are randomly and irregularly distributed throughout a matrix.
Weight Sparsity	A form of sparsity in neural networks that is typically static, resulting from model optimisation techniques like pruning.

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence

Abbreviation	Full Form
ASIC	Application-Specific Integrated Circuit
CNN	Convolutional Neural Network
DPR	Dynamic Partial Reconfiguration
DSP	Digital Signal Processor
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
GEMM	General Matrix-Matrix Multiplication
GOPs	Giga Operations Per Second
HPS	Hard Processor System
IoT	Internet of Things
IP	Intellectual Property
MAC	Multiply-Accumulate
MIF	Memory Initialisation File
NPU	Neural Processing Unit
PE	Processing Element
ReLU	Rectified Linear Unit
RTL	Register-Transfer Level
TPU	Tensor Processing Unit
UART	Universal Asynchronous Receiver-Transmitter

List of Figures

1. Introduction

Artificial Intelligence (AI) has marked significant milestones through numerous technological domains, with prominent trends in the migration of AI inference from cloud-based systems to resource-constrained processors at the edge. These applications at the edge may involve Internet of Things (IoT) sensors, mobile devices, and other embedded systems. This particular shift towards applications at the edge requires a demand for real-time responsiveness whilst having a reduced reliance on network connectivity. This presents the challenge of having to execute computationally intensive models with an adequate level of accuracy while operating within hardware resource and energy constraints for the selected embedded platforms.

This challenge has shown the presence of modern Convolutional Neural Networks (CNNs), which have become the standard for visual operations. The architectural design of these networks relies on a large number of repetitive mathematical operations that general-purpose processors are insufficient in performing.

To account for such challenges, specialised hardware accelerators (e.g. Neural Processing Units) have been developed specifically for offering performance with efficiency when compared to typical general-purpose processors. However, they are designed as a fixed architecture, which poses an inflexibility for situations and scenarios that involve neural networks containing sparsity, which can be directly induced from model optimisation techniques. A fixed-function accelerator cannot exploit this sparsity, wasting a significant amount of cycles and a proportional amount of energy performing useless computations involving zeros. This motivates a solution aimed at the investigation and development of a reconfigurable Neural Processing Unit (NPU) on a Field-Programmable Gate Array (FPGA) that can adapt to the dynamic and sparse nature of modern CNNs to maximise both computational and energy efficiency. The main Objectives include:

- To design and implement a reconfigurable NPU with a systolic array architecture as the core parallel processing engine for matrix multiplication.
- Investigate and focus on a software-based preprocessing approach for condensing structured sparse matrices into dense equivalents for increasing the efficiency of computation.
- Validate the correctness of the accelerator design through extensive simulation, where the performance can be analysed.
- Utilise the reconfigurable nature of the De1-SoC Cyclone V FPGA to prototype the accelerator and analyse the physical performance and resource utilisation.

This report details the investigation and development of an NPU. Section 2 provides a review of the relevant background literature on CNNs, edge acceleration, systolic arrays and sparsity handling, establishing the foundations of the research scope. Section 3 outlines the attempted methodology, detailing the achitectural design, software pre-processing algorithm, systolic array operation, and the verification strategies. Section 4 presents the simulation and hardware results, quantifying the qualitative performance of both unoptimised and optimised architectures. Section 5 discusses the interpretation and significance of the results to evaluate the choices made and to acknowledge the limitations presented. Section 6 discusses the proposed future work based on the limitations and failed attempts made during the process of this project.

2. Background/Literature Review

With an emphasis on systolic array optimisation and FPGA implementation, this literature review study delves into the current implementations for the design and its required background knowledge. This review highlights current gaps for more research by examining a number of recent studies.

2.1 Convolutional Neural Networks (CNN)

Convolutional Neural Networks (CNN) are a certain set of models within the category of deep learning that learn to handle grid-like data input by applying filters. CNNs are unique among other neural networks as neurons within them are connected to small regions within the same layer rather than connecting to the entire layer and across layers. These regions are called inputs.

A CNN model contains a set of layers that each have an important role:

- **Convolution Layer:** The convolutional layer performs the bulk of the model's computation. Performs a convolution, which is when a set of filters shifts across the input and computes dot products for each location. This particular process consists of a large number of Multiply-Accumulate (MAC) operations; some studies even show that because of this, the convolution layer itself may take 90% of the runtime [1], [2]. The high parallel workload can be mathematically expressed as a General Matrix-Matrix Multiplication (GMM) problem, essentially transforming the input feature maps and convolutional filters into two large matrices, allowing the entire operation to be performed as a single, highly efficient matrix product, bringing in the potential for acceleration hardware architectures that are designed for parallel matrix algebra [3].

$$ActivationMap = Input * Filter \tag{1}$$

$$\sum_{y=0}^{columns} (\sum_{x=0}^{rows} Input(x-p, y-q) Filter(x, y)) \tag{2}$$

- **Activation Layer:** Following the convolution process comes the activation layer, where an activation function is applied to allow the neural network to adapt to more complex data. It adds non-linearity to the network by calculating the weighted sum of inputs and applying a non-linear function to that sum [4]. It determines the firing of each neuron as logic 1 or 0, representing on or off. However, almost all the time, it falls to 0, which would pass zeroes through backpropagation, inevitably causing loss of information. The Rectified Linear Unit (ReLU) function is the most commonly used activation function that introduces a solution for this issue, where it gives a zero value in the presence of a negative or zero-valued input, but returns the regular value for positive inputs [5] [6].

$$f(x) = \begin{cases} x, x > 0 \\ 0, x \leq 0 \end{cases} \tag{3}$$

- **Other Layers:** The remaining layers of a CNN model discussed in [5] are the pooling layer, which combines convolutional layers for down-sampling, and the fully connected layer, which is the output of the final convolutional and pooling layer, where data is

flattened before passing through. However, they are not as relevant to this research compared to the convolutional and activation layers.

2.2 Hardware Acceleration at the Edge

There is a demand for a powerful CNN model inference, but with the limitations of resources at the edge. This poses a challenge when dealing with intensive deep learning models, as unlike cloud environments with vast computational power resources, these edge platforms are restricted by limited hardware and other memory resources. To make large CNN models more viable on devices based on the edge, the goal is to reduce the computational footprint before they run on any hardware. This can be done with various optimisation techniques that create more efficient models with minimal loss in accuracy. Two common techniques are quantisation and pruning.

2.2.1 Quantisation

Quantisation is the process of reducing the model's weight and activation numerical precision. For example, the standard 32-bit floating point representation can be converted to 16-bit or even 8-bit integers, which are more commonly used for edge-based devices. Demonstrated in a research study on optimisation [7], a 75% reduction in memory bandwidth requirements can be observed when moving from 32-bit floats to 8-bit integers, as it simplifies the arithmetic and uses less complex hardware logic.

2.2.2 Pruning

In contrast, pruning reduces the model's complexity by removing redundant parameters, such as weights that may have only a minor impact on the model's outcome. By removing these parameters, the required MAC operations can be reduced, leading to faster processing [7]. Several pruning methods were discussed in the paper, such as structured, unstructured, iterative, one-shot, weight, filter, and layer pruning, all of which tackle the same problem in different ways and intensities.

2.2.3 Device Selection

Aside from model optimisations, the choice of hardware platform is equally important for meeting the constraints at the edge. There are several choices, such as GPUs, ASICs, and FPGAs, as they provide several benefits when deploying the accelerator at the edge [8]. Application Specific Integrated Circuits (ASICs) are fixed hardcore processors as they are fixed in silicon, whereas FPGAs can provide hardware-level reconfigurability, allowing for a more customised approach. This flexibility is worth exploring when it comes to adapting hardware constraints for specific neural network model characteristics, examples of which can also be observed in [9]. Furthermore, FGPAs offer more control over hardware that cannot be achieved in more general-purpose processors. A particularly unique approach shown in [10], displays this quality as maximising the use of Look-Up Tables and on-chip memory blocks to load and store pre-computed convolutional results. This method of utilising the FPGA achieves an energy efficiency of 388 GOPs/W, making the FPGA a more feasible device for both research investigation and accelerator deployment when dealing with AI at the edge.

2.3 Systolic Array architecture

As established in the previous section, the convolution operation constitutes the most computationally intensive stage of the CNN model, involving numerous MAC operations. This computation can then be mathematically mapped into a General Matrix-Matrix Multiplication (GEMM), done

by mapping the image to a matrix of activations and mapping weights to a filter matrix. It is typically performed by the im2col algorithm, an example of which can be viewed by [11]. This algorithm allows the model to be converted to a matrix-based execution. Regular processors are limited by memory bandwidth and irregular data reusage, leading to severe inefficiencies during inference. To address this, systolic arrays can be used to map the GEMM structure directly into hardware by sequencing a pipelined movement of data by distributing the MAC units rhythmically. Such architectures have proven to be exceptionally effective for CNN acceleration, evidence of which can be observed by Google's Tensor Processing Unit (TPU) achieving 83x and 29x more performance-per-watt improvements over traditional CPU and GPU counterparts, respectively [12].

As observed in Figure 1, a systolic array consists of a grid of Processing Elements (PEs). Each PE contains a Multiply and Accumulate unit (MAC) with internal registers. The architecture's name is derived from its operational similarity to the human circulatory system, as the data are rhythmically "pumped" through the array of PEs in a pipelined manner, much like the blood through the heart [13], [14].

There are several design considerations required when creating the systolic architecture, such as the data flow or even the structure itself. Common dataflows include input-stationary, weight stationary, output stationary, and a few examples of systolic structures are linearly connected, hexagonally connected, and orthogonally connected [9], [13], each of which influences its own trade-offs between memory bandwidth and PE utilisation. For instance, output-stationary designs minimise the partial sum movement, while weight-stationary designs emphasise local reuse of weights and inputs.

$$C[i][j] = \sum_{k} (A[i][k] \times B[k][j]) \tag{4}$$

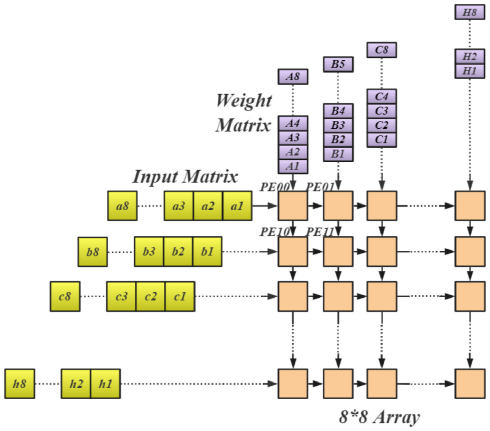


Figure 1 An Orthogonal, output-stationary systolic array architecture. [15]

Utilising a systolic array architecture has several advantages when inferencing at the edge, such as high throughput and energy efficiency through the introduction of parallelism. This parallelism is accomplished by having every PE active at every clock cycle, allowing a scaling of double, leading to a design with a latency that is also approximately doubled [16]. The energy efficiency comes from the lack of the need to read matrix element values from memory, as the data is passed sequentially along the PEs through their internal registers. This high degree of data reuse and lockstep flow reduces the memory bandwidth by a factor of its own architecture width, due to the decreased memory accesses.

2.4 Sparsity in Systolic arrays

While systolic arrays provide exceptional throughput for dense matrix multiplications, their lockstep data flow produces a significant utilisation issue if and when the inputs are not uniform. This is because the performance is based upon the assumption that all computations performed are useful. Additionally, this "uniform" data flow becomes inefficient in the presence of sparsity, which introduces zero-valued operands into the computation. When activations or weights are zero, PEs still perform redundant MAC operations, resulting in degraded utilisation and energy from hardware resources [17].

The occurrence of sparsity is caused by two main factors. The first is activation sparsity, primarily induced by activation functions such as the ReLU, which output zeros for all negative inputs. This form of sparsity is dynamic, as the distribution of zero activations varies between different input images. [18]. The second is weight sparsity, introduced through model compression techniques such as pruning, eliminating connections with negligible contribution to the model's accuracy. This sparsity is static and predictable, as the zero pattern remains constant during inference [19], [20].

Studies show that unstructured sparsity may reach up to 90% in CNNs, [18]. The unstructured sparsity may be caused by individual weight removals or the dynamic sparsity in ReLU, but it conflicts with the structured flow of systolic arrays, leading to a minimised PE utilisation and a reduction in throughput. To tackle this, multiple approaches in specialised architectures have been proposed. Firstly, the Sparse-TPU proposes a packing method that condenses the sparse matrix before sending it out to the systolic array, achieving up to 16x performance boost and a 4.39x lower energy consumption for 8-bit integer computation when compared to their baseline TPU [17]. A similar approach, such as the SCNN accelerator, uses a more novel approach to the conventional dataflow, where both weights and activations are kept in compressed formats throughout the computation, completely altering the convolution as series of outer products, where non-zero values are multiplied by an array of other non zero values, essentially allowing the hardware to skip all the unnecessary computations entirely. This approach achieved a 2.7x performance increase across several common CNN models compared to a generic accelerator as a baseline comparison [21].

To make a fair comparison of research, a parallel set of research was done for structured sparsity. For example, the Sense accelerator makes use of a load-balancing approach to ensure the weight sparsity is evenly distributed, achieving up to 2.5x speedup over the standard systolic array baseline [18]. Another implementation involves a co-design approach where the pruned matrix sizes are matched with dimensions of the systolic array [20], allowing the hardware to skip blocks of computations, achieving up to 44% speedup. More dynamic scheduling methods have also been proposed, such as the Dataflow Mirroring, which enables more fine-grained multitasking and then running a second independent task on other PEs if and when idle with a reversed dataflow. This method of maximising the use of idle PEs was shown to increase performance by [22].

Beyond the previously discussed pre-processing methods, a simpler approach is hardware-level zero-skipping logic, where the accelerator can identify the zero values and skip the MAC operation upon identification. The architecture described in [23] implements a fine-grained approach where the multiplication is disabled when the input is zero. This approach achieved 13.6 TOPs/W energy efficiency and a MAC utilisation of 83.8%. Similarly, yet a completely different strategy has been accomplished by [24], achieving a more structured zero-skipping method for FPGAs. Their accelerator does not check fine-grained zeros but instead skips computations for slices if all are zeros. Their prototype achieved an energy efficiency of 51.96

GOPS/W.

2.5 Research Gap and Project Scope

Based upon the literature review, the general consensus is that systolic arrays are highly efficient for dense computations. However, they are not suitable by nature for heavily sparse workloads common in most CNNs. The research carried out a pursuit to discover the reasoning behind these unwanted inefficiencies and discovered that either highly specialised hardware was required or software-hardware co-designs were obtained in order to take into account sparse inputs. Both approaches present significant trade-offs that create a clear opportunity for a more unique methodology. Architectures like SCNN and Sparse-TPU handle unstructured sparsity, which demonstrate impressively high performance boosts. However, this comes at the cost of significant complexity in hardware for managing the irregular data, which may end up having more power consumption than required when deploying an FPGA at the edge. Furthermore, this complexity strays from what makes a systolic array a simple architecture. On the other hand, simpler approaches, such as the hardware zero-skipping logic, lacked the flexibility of an all-rounded solution since it would only be effective for reducing power consumption and not reducing the throughput, as the data would still need to traverse through the PE slot at hand. This analysis allowed for the identification of a research gap for a more flexible hardware/software approach. The desired approach was to take the hardware simplicity of a regular output-stationary systolic array whilst tackling the unwanted sparsity. The following work explores a practical and more resource-efficient methodology for accelerating CNNs at the edge, prioritising hardware simplicity and reconfigurability.

3. Methodology

3.1 Architectural Design and Pipeline

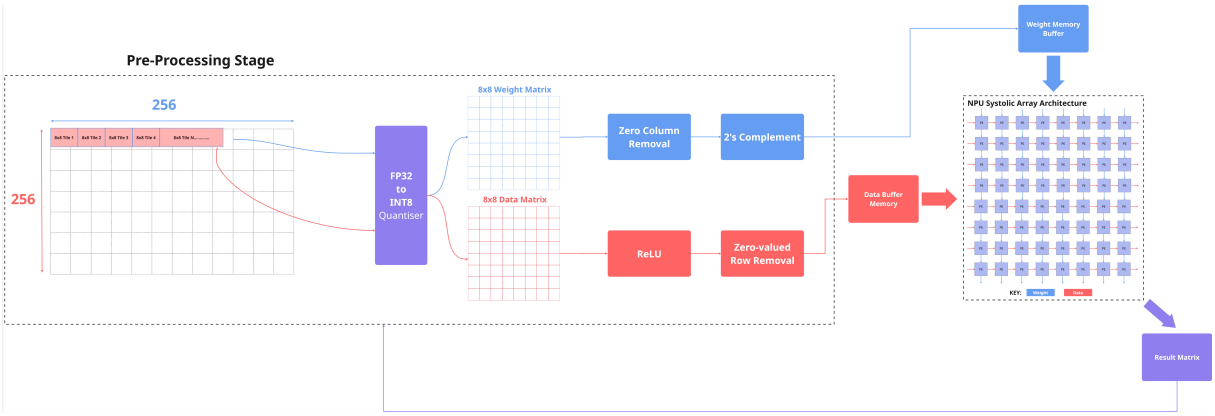


Figure 2 High-level System Diagram showcasing the HW/SW Co-design topology

The design of the full system architecture observed in Figure 2 is discussed here in a pipelined manner:

- **Python Preprocessing using AlexNet:** A software-based preprocessing stage is used to extract the weight and activation matrices from a pretrained AlexNet model and apply the tiling method. The Python script also handles structured sparsity by identifying and removing rows and columns that entirely contain zeros, based upon mathematical conditions discussed further in Section 3.2.2. This generates smaller, denser matrices

alongside their new dimensions: *active_rows*, *active_cols*, *active_k*, serving as inputs to the hardware for further processing.

- **Data and Weight Buffer Memory:** The on-chip BRAM component serves as the buffer memory for the accelerator, which stores the dense arrays provided by the preprocessing stage via memory initialisation files (MIF). This local storage on the FPGA is required to feed the hardware at an optimal speed by avoiding potential memory access latencies.
- The systolic array architecture can be described as a top-down pipeline, with components such as:
  - **Control Unit:** The systolic array control unit is a counter-based controller that simply orchestrates the entire matrix multiplication process based upon the *active_rows*, *active_cols*, *active_k* parameters read from the buffer memory. It uses a counter to generate the precise, staggered timing signals required for the data and weight shift register arrays. Concurrently, it produces a static PE enable mask to ensure that only the PEs within the active matrix dimensions are enabled for the duration of the operation, which is critical for reconfigurability and power optimisation.
  - **Systolic Array:** This is a structural entity that instantiates the processing elements in a reconfigurable 8x8 grid and routes the shift register arrays to the top and left edges, respectively. The systolic array internally produces bus signals to propagate the data left to right and weights top to bottom between the adjacent PEs.
  - **Processing Element:** The core component of the accelerator, designed as a pipelined array of MACs, each of which contains an 8x8-bit signed multiplier, in which the 16-bit result is added to the 32-bit accumulator register to prevent overflow. Each PE's operation is governed by an enable signal bit, allowing unused PEs to be gated, and its final accumulated value is held in the result register

3.2 Software Pre-Processing and Sparsity Handling

The goal was to maximise the computational efficiency and energy efficiency where possible. The systolic array architecture is inherently inefficient when processing sparse data, as it cannot bypass useless computations involving zero-valued elements itself. Consequently, a hardware/software co-design approach was adopted, where the software pre-processing stage is responsible for transforming a sparse matrix into an equivalent, smaller dense matrix that the hardware is optimised to compute. This particular approach specifically targets structured sparsity, which displays itself as entire rows and columns of zeros, mostly in the weight matrix due to pruning, but also due to the activation functions applied to the image inputs.

3.2.1 AlexNet Data Extraction and Tiling

To accurately make the connection from such a high-level neural network to low-level hardware, a multi-stage software pipeline was developed. The process began by loading a pre-trained model from the PyTorch library using the AlexNet, as it is widely used as a standard benchmark in research papers discussing hardware acceleration at the edge [18], [25]. It was then quantised to convert its activations from 32-bit floating point to an 8-bit signed integer representation. Quantisation is important to ensure hardware efficiency, as it reduces memory bandwidth by a factor of four and allows for less complex arithmetic on the FPGA, as discussed in Section 2.2 [7]. Furthermore, the necessary tensors for the layer at hand were extracted after applying the activation function, ReLU. The weight integer was also converted to a two's complement binary format to match the signed data type used in hardware, as there may be negative-valued weights present.

The matrices for a given convolution layer in the AlexNet were very large, reaching sizes of (3025, 576) and (64, 363) for activation and weight matrices, respectively. A hardware accelerator would need an extremely large number of PEs, resources of which are not adequate in most development board FPGAs. The primary resource constraint for the target development board was the limited DSP blocks required for MAC operations within each PE, containing only 87 blocks. To fit within this constraint, an 8x8 PE grid was selected, requiring only 64 blocks for the accelerator computations. For this to work correctly, the large matrices collected from the convolution layer were sliced into smaller 8x8 matrices using the tiling method, allowing for a more resource-efficient hardware accelerator. All the following preprocessing is required before sending to the systolic array, which is applied to these 8x8 tiles. To ensure an even divisibility when dividing into 8x8 tiles, the matrices were zero-padded to ensure that they could be divided into 8x8 tiles evenly.

3.2.2 Sparsity-Handling Algorithm

The pre-processing algorithm utilised the simple mathematical rule of the standard matrix multiplication formula for a given layer:

$$C[M * N] = A[M * K] * B[K * N] \tag{5}$$

As observed in Equation 5, A represents the data matrix, B represents the weight matrix, and C is the resulting output. The dimensions M and N define the spatial dimensions of the output, whilst the common inner dimensions, K, dictate the computational depth of the operation [26], [27]. The algorithm then analyses both matrices to identify any rows or columns that consist entirely of zero-valued elements. This is achieved by creating a Boolean mask of the computationally relevant rows and columns, and by using these masks, the original matrices can be compacted down by removing any row or column. With this compaction, smaller, denser matrices can be produced. The algorithm follows the rule of matrix multiplication shown in Equation 5, where when a column is removed from the activation matrix, the corresponding row must be removed from the weight matrix to maintain the mathematical validity of the formula.

The output of this stage is divided into the compacted dense matrices and a set of integer parameters representing the reduced dimensions (M, N, and K). These parameters can be stored in memory alongside the matrices, where the parameters can then be passed onto the control unit for the accelerator, enabling it to reconfigure the systolic array to the exact size and execute only the necessary number of MAC cycles, ultimately increasing the computational efficiency.

3.3 Systolic Array Operation

3.3.1 Systolic Array and its Processing Elements

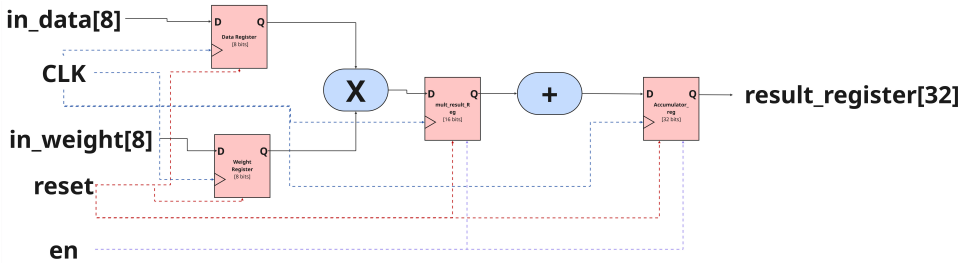


Figure 3 RTL Diagram of Processing Element

As displayed in Figure 3, the PE was designed as a MAC unit where each PE consists of two 8-bit signed multipliers, a 16-bit multiply register, and a 32-bit accumulator register for

preventing any overflow during accumulation. Each PE receives data and weight values and passes them to adjacent PEs, essentially forming a grid of the systolic array.

These PEs were instantiated within the systolic array entity to form the 8x8 2D systolic array as per the requirements discussed in Section 3.2.1. Within this grid, the data output of each PE is connected to the data input of the adjacent PE to the right, while the weight output is connected to the weight input below it, creating the orthogonal data path for propagation of both matrices [13]. For this project, an output-stationary design, observed in Figure 4, was selected, where the partial sums form the stationary final matrix. This decision leads to a more simplified PE design as the PE's core logic has its own MAC unit that adds the product of its current inputs to its own accumulator register without requiring any additional input to receive partial sums from the adjacent PEs. A more balanced input approach like this to the systolic array allows for an increase in data reuse for every PE, essentially reducing the demand on memory [18]. The functional correctness of an output-stationary systolic array depends on the precisely orchestrated flow of data. For a given PE, to correctly compute its output value, the corresponding input data and weight and input data elements must arrive at its inputs at the exact same clock cycle. This requires a higher-level controller to stagger the data into the array's boundaries, creating a diagonal wavefront of computation. The control unit in this project is designed specifically to generate this timing pattern.

Figure 4 Systolic Array Processing

3.3.2 Control Unit Staggering Logic

The control logic is implemented using a count-based approach. This takes into account two details. Firstly, the first element of row i of the data matrix is placed into the array at the beginning of the first clock cycle i. Subsequent elements for that row are then placed on each following clock cycle. Secondly, the first element for column j of the weight matrix is placed into the array at the start of the clock cycle j. The staggering will ensure that as the data propagates horizontally and the weights propagate vertically, they meet at the correct PE at the correct time.

The integration of M, N, and K modified the baseline staggering logic in multiple ways. For sparsity handling, there were internal conditional checks that use those parameters to ensure that only the rows and columns corresponding to the compacted matrix dimensions are actively loaded from the buffers into the shift registers and rows beyond are fed zeros. The computational depth (K) parameter determined the duration for which valid data was fed into each row and column. Extra conditional statements ensured that the data elements were only loaded from the buffers during clock cycles required for K computations. Once the counter exceeds the necessary window for a given row/column, the control unit feeds zeros into the corresponding

shift register element for the remaining cycles to prevent unnecessary data propagation.

Concurrently, it produced a PE enable mask using these parameters at the start of the operation. This mask identified the rectangular region of PEs required for the compacted computation. Only PEs within this region receive an active enable signal, details of which can also be observed in 3. PEs outside this grid are effectively gated for the entire duration, preventing unnecessary power consumption.

3.4 Hardware Scalability and Run-Time Reconfigurability

An important feature of the systolic array's design was the scalability at synthesis time. The structure of the PE grid is defined in VHDL using generate statements, which programmatically instantiates the PEs and their interconnections based on a generic parameter. This approach was chosen as it creates a more parameterisable design, allowing the array's physical dimensions to be modified by the designer at synthesis time. Layered on top of this fixed physical grid is the run-time reconfigurability, essentially the core of the accelerator's adaptability. The control unit reads the dimensions produced by the software stage and generates a static boolean mask at the beginning of an operation. This bit mask is distributed to the entire array, and each PE is designed to only be enabled when its corresponding bit in the mask is active. This is a critical design decision for power efficiency, as it allows unused PEs in the physical grid to be gated, preventing unnecessary switching activity and significantly reducing the accelerator's power consumption when processing matrices smaller than the 8x8 hardware capacity.

3.5 Verification and Validation Strategy

To ensure that the hardware design operates correctly at the Register-Transfer Level (RTL) prior to synthesis, the verification was executed within ModelSim. This allowed for almost a self-checking workflow that integrated the Python pre-processing stage with a custom VHDL testbench, designed to validate the entire accelerator pipeline from memory input to final result output. The Python script first sent the tiled and processed matrices using the sparsity-handling algorithm, and also calculated the correct final output matrix using Numpy, which was used for comparison when analysing the simulation waveforms. The dense input matrices and their dimensions were then converted into Memory Initialisation Files (.mif) to be loaded into the ROM component designed by the Quartus IP Catalog, which is then read by the VHDL testbench, accurately simulating how the hardware would fetch data from the on-chip memory source on the FPGA. When taking the theoretical latency calculations, the results can be cross-referenced with expected results from MATLAB and Python.

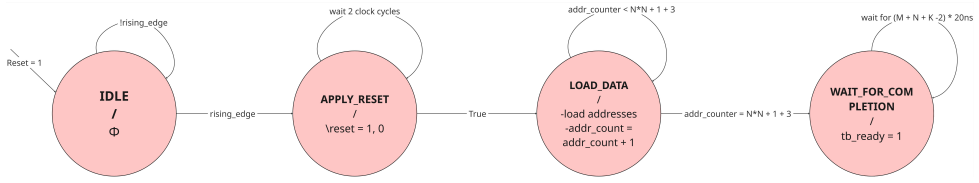


Figure 5 Testbench ROM Reading FSM

The main testbench was designed to verify the reading of each tile from the buffer memory. It is controlled by a Finite State Machine (FSM), executing a memory read and store. Illustrated in Figure 5, the FSM transitions through a sequence of states to manage the test. From a reset state to ensure determinacy and a load state to ensure that the ROM contents are sequentially read. Once all inputs are provided, a transition to the waiting state is made to wait for the accelerator to compute the matrix, all the while calculating the latency of the systolic array

operation. Upon completion of the simulation, the result matrix can be compared with the reference matrix produced by the Python script, where a significant latency reduction and match in numbers provide confidence in the functional accuracy of the design.

3.6 FPGA Hardware Validation

Following successful verification in simulation, the current implementation was synthesised to attempt to collect resource utilisation metrics. The accelerator was implemented using the Intel Quartus Prime software, targeting the Cyclone V DE1-SoC development board. To validate the timing constraints, a minimalist hardware test system was developed. In this system, the input matrices were preloaded onto the on-chip memory at synthesis time, and the computation was initiated via a button click.

3.6.1 Hardware Wrapper and UART

The top-level entity, `de1_soc_top`, serves as the physical interface for the system as it is responsible for connecting the design to the DE1-SoC board ports, mapping the 50MHz on-board clock, and connecting the push buttons with a debouncer and UART TX/RX pins. This module instantiates the core NPU controller for synthesis, the `NPU_wrapper`.

The `NPU_wrapper` entity consists of the central controller of the hardware testing, where it determines the sequencing of the entire process via FSM logic, displayed in Figure 6. This FSM is designed to execute a pre-defined matrix multiplication upon receiving a start signal from a push button, sequencing through a series of states. It first enters **`S_LOAD_PARAMS`** state to read the matrix dimensions from ROM files. It transitions to **`S_LOAD_MATRICES`**, sequentially reading the memory content and storing it. Once the data is loaded, the FSM enters the **`S_EXECUTE`** state, where the ready flag is asserted and enters a waiting state, **`S_WAIT_DONE`**. This last waiting state is where the systolic array can begin its computation and keep in check with a latency cycle counter calculated by the formula: (M + N + K - 2). Upon reaching the end of the cycle counter, exceeding the latency, a done flag is pulsed. This pulse is to indicate completion of the matrix multiplication, and the latency is transmitted off-chip via an integrated Universal Asynchronous Receiver-Transmitter (UART) module. The serial data was captured using a Raspberry Pi Pico as a USB-to-serial interface and displayed on a computer terminal. This hardware served as a stepping stone to a more streamlined pipeline in future, confirming that the timing logic works correctly for the accelerator. The Raspberry Pi is programmed using MicroPython to receive all the UART signals at the standard baud rate of 9600 bd. The protocol data packet consisted of a 10-bit input, 8 of which were data bits, 1 start bit and 1 stop bit

This testing process is used both for testing the unoptimised and optimised systems. The main difference is that the unoptimised system involves the hardcoding of matrix values through a testbench, where the control unit will be able to take them in order to produce the staggering logic. On the other hand, the optimised system involves ROM memory loading, where the Python pre-processing stage will generate the MIF files and store them in these ROMs. And from the FSM, the NPU will wait for a start signal to start in a proper sequence.

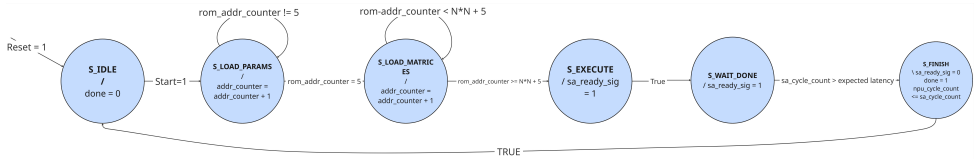


Figure 6 Testbench ROM Reading FSM

4. Results

This section presents the findings of the quantitative approach from the simulation and hardware synthesis of the project. The evaluation is based on the comparison of performance between the optimised sparsity-aware architecture and an unoptimised fixed-size architecture.

4.1 Simulation Results

4.1.1 Performance of Unoptimised Hardware Accelerator

The data presented in Figure 7 demonstrates a predictable relationship between the matrix dimension, N, and the computational latency, measured in clock cycles. The smallest NxN started off with a 1x1 operation, which had a latency of 1 clock cycle. As the matrix increased in size, the latency grew proportionally, with a 2x2 requiring 4 clock cycles, a 4x4 operation requiring 10 clock cycles, and a full 8x8 operation taking 22 clock cycles. These results from the simulation matched with the theoretical latency formula of 3N-2 for a square NxNxN multiplication on this particular Output-Stationary architecture.

Size of Matrix A and Matrix B	Latency
1x1	2
2x2	5
3x3	8
4x4	11
5x5	14
6x6	17
7x7	20
8x8	23

Figure 7 Baseline Systolic Array latency On Different NxN Sizes

Size of Matrix A	Size of Matrix B	Latency
1x8	8x8	23
2x8	8x8	23
3x8	8x8	23
4x8	8x8	23
5x8	8x8	23
6x8	8x8	23
7x8	8x8	23
8x8	8x8	23

Figure 8 Latency Based on Varying Input Matrix Sizes

In order to validate the timing model of the baseline accelerator, a second set of tests was conducted using dense, non-square matrices. The purpose of this experiment was to confirm that the hardware's latency is determined by the physical dimensions of the array when the sizes are varied. The results for several rectangular matrix configurations are illustrated in Figure 8. As shown in the data, the measured latency for all test cases was 23 clock cycles.

4.1.2 *Performance of Optimised Hardware Accelerator*

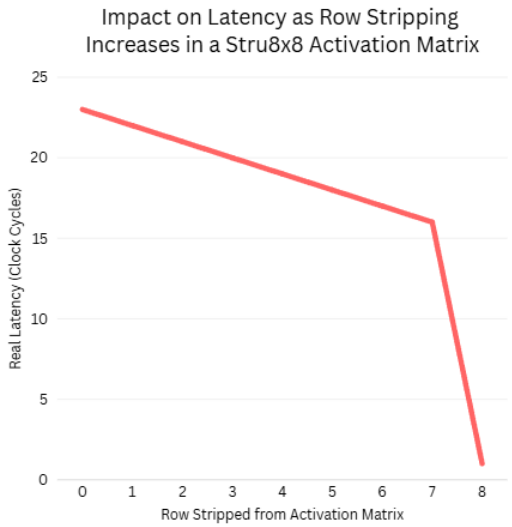


Figure 9 Impact on Latency as Row Stripping Increases in a 8x8 Activation Graph

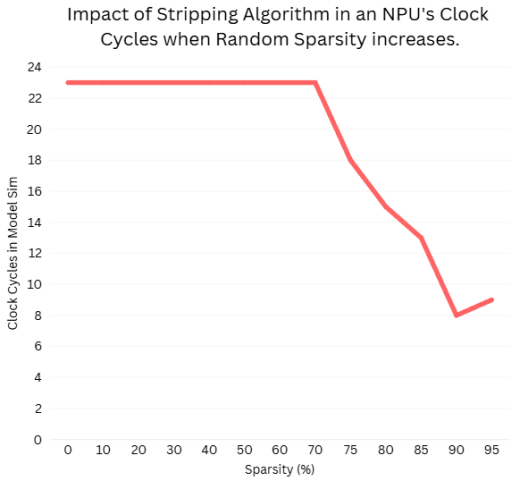


Figure 10 Figures/Impact of Stripping Algorithm on Clock Cycles when Applied on Random Sparsity Graph

The graph presented in Figure 9 illustrates the impact of the sparsity-handling algorithm on computational latency. The trend shows that for a dense 8x8 matrix where no rows are stripped, the latency is 23 clock cycles. As the algorithm identifies and zero-valued the rows, the latency decreases. A linear reduction can be observed as it reaches 7 rows stripped from the input matrix, whilst the weight matrix consists of a fully dense grid. Consequently, when all 8 rows are identified as zero value and then removed, the latency drops to 1 cycle, effectively skipping the computation. Similarly, the graph presented in Figure 10 also demonstrates the effectiveness of the stripping algorithm. Specifically, it plotted the latency against the percentage of zero-value elements randomly distributed within the 8x8 tiles. It demonstrates almost like a threshold effect at 70% , with random sparsity levels below 70%, there seems to be no noticeable effect. However, once random sparsity exceeds the 70% mark, the number of rows and columns that become zero increases significantly, and a sharp decrease in latency can be observed. The maximum reduction can be observed at 90% sparsity, where latency can be seen dropping to approximately 8 cycles. This represents a reduction of up to 65% compared to the baseline, once again, highlighting the algorithm's effectiveness even when sparsity is not perfectly structured initially.

Tile	M	N	K	Active PEs (M*N)	Latency (Real)	Difference from Baseline
Tile 0	0	8	8	0	1	22
Tile 1	1	8	8	8	16	7
Tile 2	2	8	8	16	17	6
Tile 3	3	8	8	24	18	5
Tile 4	4	8	8	32	19	4
Tile 5	5	8	8	40	20	3
Tile 6	6	8	8	48	21	2
Tile 7	7	8	8	56	22	1
Tile 8	8	8	8	64	23	0

Figure 11 Effect of M value on the number of Active PEs in the Systolic Array and Latency

The latency reduction is a direct consequence of reduced PE utilisation. Figure 11 details

the correlation between the number of active rows and the number of active PEs. In the baseline case where M=8, all 64 PEs are active. As M decreases, the number of active PEs reduces linearly. For instance, when the algorithm determines that only two rows are active (M=2), the PE utilisation drops by 75% to 16 active PEs. This correlation demonstrates the effectiveness of the added sparsity handling algorithm during the Pre-Processing stage.

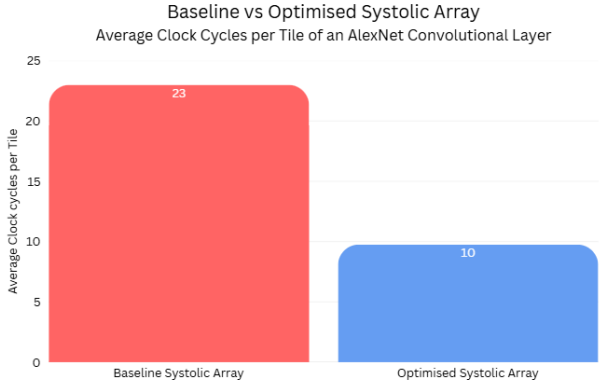


Figure 12 Average Clock Cycles per Tile of an AlexNet Convolutional Layer Comparison

To evaluate the optimised NPU's potential further, the performance was benchmarked against a fixed-size architecture. For this, a Python simulation function was used to process 156 images in one go, specifically chosen to represent a wide variety of sparsity levels encountered in several practical applications, including medical X-rays, landscapes, and cats. This script precisely calculated the effective M, N, and K dimensions for each 8x8 processing tile after applying the sparsity algorithm, allowing for an accurate latency simulation. The findings of this data collection is displayed in Figure 12. This bar graph contrasts the average computational latency per tile for the two varying architectures. The baseline systolic array consistently required 23 clock cycles to process each 8x8 tile, regardless of the content being fed. On the other hand, the expected calculation for the optimised NPU had an average latency of 10 clock cycles per tile across the entire dataset. This represents a significant average performance improvement, quantified as a 60.86% reduction in latency for each tile processed. It is important to note that this average incorporates both highly sparse tiles and denser tiles, reflecting more realistic operation efficiency. Evidently, the impact of this per-tile saving becomes clear once extrapolated to the scale of a complete model like AlexNet. Using the calculated 0.71 GMACs for AlexNet (710 million MAC operations), the 60.87% latency reduction translates into a theoretical saving of over 432 million clock cycles for every single inference. This reduction in required computation corresponds directly to an estimated overall performance speedup of 2.55x (1 / (1 - 0.6087).

4.2 Hardware Synthesis Results

Following the simulation-based evaluation, the NPU design was synthesised to determine its physical performance characteristics. Timing analysis was performed on the synthesised design, which successfully met all setup and hold time constraints, indicated by a positive slack of +6.543 ns for setup and +0.308 ns for hold. This confirms the stability of the design and validates the reported maximum frequency of 289.27 MHz for the slow 1100mV 85C model.

Using this fmax value, the theoretical peak throughput for AlexNet can be estimated using calculations from the data presented in 12. By combining the hardware's max clock speed with the previously measured average latency reduction of 60.87%, the optimised NPU achieves a theoretical peak throughput of 1.478 GOPs/sec (Giga Operations Per Second). For comparison,

the baseline architecture operating at the same max frequency but without the sparsity handling optimisations would achieve an estimated peak throughput of only 0.579 GOPs/Sec. These results demonstrate that the proposed sparsity-aware design provides a 2.55x increase in theoretical throughput.

$$\begin{aligned} \text{AlexNet} &= 0.71 GMACs \\ 2OPS &= \text{1Multiplication + 1 Accumulation} \\ fmax &= 289.27 Mhz \\ \text{AlexNet Completion Time} &= \frac{0.71 \cdot 10^9 cycles}{289.28 cycles/sec} \\ \text{AlextNet Completion Time} &= 2.45 sec \\ time/speedupfactor &= 2.45/2.55 \\ speedupTime &= 0.9607 sec \end{aligned} \tag{6}$$

$$\frac{\text{Total Operations}}{\text{Time}_{\text{Optimized}}} = \frac{1.42 \times 10^9 \text{ Ops}}{0.0607 \text{ sec}} \approx \mathbf{1.478} \text{ GOPs/sec} \tag{7}$$

$$\frac{\text{Total Operations}}{\text{Time}_{\text{baseline}}} = \frac{1.42 \times 10^9 \text{ Ops}}{2.454 \text{ sec}} \approx \mathbf{0.5786} \text{ GOPs/sec} \tag{8}$$

Model	Fmax	Set Up	Hold
Slow 1100mV 0C	289.27 MHz	6.543ns	0.308ns
Slow 1100mV 0C	294.9 MHz	6.609 ns	0.292 ns

Figure 13 Results from Quartus Timing Analysis (only Fmax and Slack

5. Discussion

This section provides a detailed interpretation of the findings in Section 4, evaluating their significance in the context of the project scope, objectives, and literature review. The discussion analyses the effectiveness of the approach, contrasting it with the more advanced designs mentioned in the literature review. The performance gains are then critically evaluated, and the limitations are addressed.

5.1 Interpretation and Significance of Sparsity-Aware Acceleration

The simulation results discussed in Section 4 validate the effectiveness of the approach detailed in Section 3 for accelerating sparse CNN computations.

The defining factor of success is the 60.87% average reduction in theoretical latency per processing tile compared to the baseline, going from 23 cycles down to 10 cycles across a diverse 156-image dataset 12. Interpreting this average requires a clear understanding of the distribution of data, as some tiles compute closer to the baseline, and the low average latency is produced by the extremely large sparse matrices found in image backgrounds or uniform regions. This demonstrates the theoretical efficiencies on realistic, varied data.

The analysis of random sparsity further aligns with this interpretation described in Figure 10, the 70% sparsity shows that the high activation sparsity typically generated by ReLU functions in CNNs can be mitigated; however, it only seems to be most effective above the 70% mark due to the handling of structured sparsity which is more apparent after a a large amount of zero elements present. In this particular case, the rapid reduction in cycles after the 70% mark could

also be due to the fact that the 8x8 tiling produced a zoomed effect on the large matrix at hand, so any minor sections of zeros may appear to be in large quantities, once zoomed in.

The hardware synthesis results further proved this potential, validating a high maximum operation frequency of 289.27MHz, which enables an estimated theoretical peak throughput of 1.478GOPs/sec. This 2.55x increase over the baseline operation shows a clear improvement despite the simpler approach this architecture carries. Comparing these findings against the research gap identified in Section 2.5, this project theoretically creates a balance. Unlike some of the other complex architectures targeting unstructured sparsity, such as the SCNN architecture [21], which may end up being resource-intensive for FPGAs, or simple hardware gating only offering minimal power efficiency improvements [23]. Our approach achieves theoretical performance improvements that appear promising, whilst also maintaining the relative hardware complexity of a standard systolic array using a hardware-software co-design topology, a method that is backed up by the observations and findings from Section2.4, [18], [20].

Ultimately, the relevance relies on the 2.56x theoretical speedup extrapolated for AlexNet. This level of acceleration addresses the challenge of deploying resource-heavy models on resource-constrained edge devices by reducing the bottleneck introduced by AI applications.

5.2 Architectural Evaluation

The performance data detailed previously in Section 5.1 are enabled by the accelerator's core architectural basis, which is dynamic reconfigurability based upon the M, N, and K parameters derived from the software pre-processing stage. This process allows the accelerator to adapt its computation precisely to the effective dimensions of the sparse workload. The effectiveness of this approach is quantitatively demonstrated in Figure 11, showing a linear relationship between the reduction of active rows and the corresponding decrease in active PEs. For instance, when the algorithm compacts the problem to two rows, the number of active PEs drops from 64 to 16 PEs, which is a 75% reduction, contrasting to a standard systolic array, which would invariably engage all 64 PEs regardless of any presence of sparsity, thereby consuming more power. As per the details discussed in Sections 1 and 2.5, the current architecture was produced successfully to address the problem at hand, which was to handle structured sparsity using reconfigurability for the systolic array, but without changing its core functionality to minimise potential overhead.

It is important to note that both the hardware components' reconfigurability and the software's intelligence is what make the design work imperatively, both of which cannot operate without each other. The Python script provides intelligence that the accelerator cannot possess, having the ability to analyse dense matrices and extract the M, N, and K parameters. Furthermore, the control unit provided the interpretation and execution of these parameters to generate the appropriate PE enable mask and staggered timing signals, physically adapting the hardware to the compacted workload. A compacted approach is the central contribution. When compared to the alternative approaches discussed in 2.4, such as the fine-grained hardware zero skipping, which cannot reduce the overall latency, as data still needs to traverse the array. In contrast, the method eliminating entire rows and columns can provide both latency and the number of active PEs, which is critical for overall performance and energy efficiency on edge devices.

Upon testing with the data derived from AlexNet's convolutional layers, an interesting pattern was discovered regarding the source of sparsity. While the sparsity-handling algorithm was designed to compact both weight and activation matrices, the activation matrices consistently exhibited significantly more structured sparsity than the weight matrices. This was reflected in the calculated M, N, and K parameters, where the reduction in M was more pronounced than the reductions in N or K, indicating significant row stripping, as the N and K values often remained

closer to the maximum tile dimensions, further proving dense weight structures. Linking back to Section 5.1, this is most likely due to the ReLU producing sparsity that causes large regions of zeros.

5.3 Limitations

- The sparsity handling algorithm targets only structured sparsity where entire rows/columns can be removed, which means it cannot handle unstructured, fine-grained sparsity, limiting benefits when random zeros dominate. Performance gains significantly diminish below 70% random sparsity.
- The current hardware control FSM can only manage the execution of one pre-loaded tile using MIFs, lacking a higher-level pipeline required to process entire layers effectively.
- All the performance metrics are derived from simulations and test scripts. A complete, proper pipeline operating on an FPGA could introduce hardware bottlenecks commonly present at the edge, ultimately making the speedup calculations merely an optimistic estimate.
- Hardware prototyping and testing focused specifically on an 8x8 tile size;e some data collected were only from the first convolutional layer of AlexNet. Performance on other layers, different tile configurations, or other CNN models remains unevaluated due to hardware and time constraints.

6. Future Work

6.1 Software Enhancements

- Introduce a fine-grained sparsity handling system that compacts matrices when unstructured sparsity is present. This would complement the current approach by offering further improvements in energy efficiency within the active PE grid.
- Apply the AlexNet to a real-world application like live footage detection from a camera rather than just processing images.

6.2 Hardware Enhancements

- Explore and investigate the use of Dynamic Partial Reconfiguration (DPR) to be able to switch out different systolic array configurations or PE designs in and out of the FPGA fabric at runtime, allowing the hardware to adapt even more flexibly to different models or layers.
- Integrate with on-chip Processor: Connect the NPU as a hardware accelerator to a softcore processor such as NIOs II or the Hard Processor System available on the De1-SoC. This would allow for a complete pipeline capable of running the entire AI application on the FPGA.

7. Conclusion

In conclusion, the research gap for accelerating sparse CNNs on resource-constrained devices at the edge was address exceptionally well. The inefficiencies presented by fixed systolic

array architectures provided the motivation to design a reconfigurable system with the final methodology involving a hardware-software co-design approach. This leveraged both the pre-processing stage and a reconfigurable 8x8 systolic array in VHDL to execute the computation efficiently. Most of the primary objectives were met, as the NPU was designed with an adaptable control unit based upon the M, N, and K parameters. The verificiation accomplished through ModelSim simulations, driven by data provided from the AlexNet model, which validate the correctness of this approach. The key findings of this approach were a 60.87% average reduction in computational latency per processing tile across a diverse 156-image set, translating to a 2.55x performance speedup. This was backed up further by the hardware timing analysis, showcasing a maximum operating frequency of 289.26 MHz with positive timing slack. This enabled the calculation of a theoretical peak throughput of 1.478 GOPs/sec. Ultimately, this project demonstrates the co-design methodology approach, showcasing the effective mitigation of performance bottlenecks encountered with AI at the edge.

References

[1] J. Cong **and** B. Xiao, "Minimizing Computation in Convolutional Neural Networks," en, **in***Artificial Neural Networks and Machine Learning – ICANN 2014* S. Wermter **andothers**, **editors**, Series Title: Lecture Notes in Computer Science, **volume** 8681, Cham: Springer International Publishing, 2014, **pages** 281–290, ISBN: 978-3-319-11178-0 978-3-319-11179-7. DOI: 10.1007/978-3-319-11179-7_36. **url**: http://link.springer.com/10.1007/978-3-319-11179-7_36.

[2] J. Zhang **and** J. Li, "Improving the Performance of OpenCL-based FPGA Accelerator for Convolutional Neural Network," **in***Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* **jourser** FPGA '17, New York, NY, USA: Association for Computing Machinery, **february** 2017, **pages** 25–34, ISBN: 978-1-4503-4354-1. DOI: 10.1145/3020078.3021698. **urlseen** 18 **october** 2025. **url**: https://doi.org/10.1145/3020078.3021698.

[3] J. Wu, "Introduction to convolutional neural networks," *National Key Lab for Novel Software Technology. Nanjing University. China*, **journvol** 5, **number** 23, **page** 495, 2017.

[4] J.-L. Boulanger, *Certifiable Software Applications 2: Support Processes*. Elsevier, 2016.

[5] Purwono, A. Ma'arif, W. Rahmaniar, H. I. K. Fathurrahman, B. Setiawan **and** H. Wibowo, "Understanding of Convolutional Neural Network (CNN): A Review," *International Journal of Robotics and Control Systems*, **journvol** 2, **number** 4, **pages** 739–748, 2022. DOI: 10.31763/ijrcs.v2i4.888.

[6] H. H. Tan **and** K. H. Lim, "Vanishing Gradient Mitigation with Deep Learning Neural Network Optimization," **in***2019 7th International Conference on Smart Computing & Communications (ICSCC)* **june** 2019, **pages** 1–4. DOI: 10.1109/ICSCC.2019.8843652. **url**: https://ieeexplore.ieee.org/document/8843652.

[7] J. Gorvadiya, A. Chagela **and** M. Roy, "Energy Efficient Pruning and Quantization Methods for Deep Learning Models," en, **in***2025 International Conference on Sustainable Energy Technologies and Computational Intelligence (SETCOM)* Gandhinagar, India: IEEE, **february** 2025, **pages** 1–6, ISBN: 979-8-3315-2054-0. DOI: 10.1109/SETCOM64758.2025.10932458. **url**: https://ieeexplore.ieee.org/document/10932458/.

[8] K. Guo, S. Zeng, J. Yu, Y. Wang **and** H. Yang, "[DL] A survey of FPGA-based neural network inference accelerators," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, **journvol** 12, **number** 1, **pages** 1–26, 2019.

[9] Y.-H. Chen, J. Emer **and** V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," *ACM SIGARCH computer architecture news*, **journvol** 44, **number** 3, **pages** 367–379, 2016.

[10] K. Guha **and** A. Chakrabarti, "Secure Energy-Efficient Implementation of CNN on FPGAs for Accuracy Dependent Real Time Task Processing," en, **in***2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)* Knoxville, TN, USA: IEEE, **july** 2024, **pages** 1–6, ISBN: 979-8-3503-5411-9. DOI: 10.1109/ISVLSI61997.2024.00012. **url**: https://ieeexplore.ieee.org/document/10682707/.

[11] NVIDIA Corporation. "CUTLASS: Implicit GEMM Convolution." NVIDIA Developer Documentation, **urlseen** 12 **october** 2025. **url**: https://docs.nvidia.com/cutlass/media/docs/cpp/implicit_gemm_convolution.html.

[12] G. I/O, *An in-depth look at Google's first Tensor Processing Unit (TPU)*, en, **may** 2017. **urlseen** 14 **october** 2025. **url**: https://cloud.google.com/blog/products/ai-machine-learning/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu.

[13] H. T. Kung **and** C. E. Leiserson, "Systolic arrays (for VLSI)," **in***Sparse Matrix Proceedings 1978* Society for industrial **and** applied mathematics Philadelphia, PA, USA, **volume** 1, 1979, **pages** 256–282.

[14] H.-T. Kung, *Why systolic architecture?* Design Research Center, Carnegie-Mellon University Pittsburgh, PA, USA, 1982.

[15] B. Wang, S. Ma, G. Zhu, X. Yi **and** R. Xu, "A novel systolic array processor with dynamic dataflows," *Integration*, **jourvol** 85, **pages** 42–47, 2022.

[16] H. Snopce **and** A. Aliu, "Latency Analysis in the 2-Dimensional Systolic Arrays for Matrix Multiplication," en, *International Journal of Computers*, **jourvol** 15, **pages** 1–7, **march** 2021, ISSN: 1998-4308. DOI: 10.46300/9108.2021.15.1. **url**: https://www.naun.org/main/NAUN/computers/2021/a022007-001(2021).pdf.

[17] X. He **andothers**, "Sparse-TPU: adapting systolic arrays for sparse matrices," en, **in***Proceedings of the 34th ACM International Conference on Supercomputing* Barcelona Spain: ACM, **june** 2020, **pages** 1–12, ISBN: 978-1-4503-7983-0. DOI: 10.1145/3392717.3392751. **url**: https://dl.acm.org/doi/10.1145/3392717.3392751.

[18] W. Sun, D. Liu, Z. Zou, W. Sun, S. Chen **and** Y. Kang, "Sense: Model-Hardware Codesign for Accelerating Sparse CNNs on Systolic Arrays," en, *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, **jourvol** 31, **number** 4, **pages** 470–483, **april** 2023, Publisher: Institute of Electrical and Electronics Engineers (IEEE), ISSN: 1063-8210, 1557-9999. DOI: 10.1109/tvlsi.2023.3241933. **url**: https://ieeexplore.ieee.org/document/10043636/.

[19] S. Han, J. Pool, J. Tran **and** W. Dally, "Learning both weights and connections for efficient neural network," *Advances in neural information processing systems*, **jourvol** 28, 2015.

[20] P. Palacios, R. Medina, J.-L. Rouas, G. Ansaloni **and** D. Atienza, "Systolic arrays and structured pruning co-design for efficient transformers in edge systems," **in***Proceedings of the Great Lakes Symposium on VLSI 2025* 2025, **pages** 320–327.

[21] A. Parashar **andothers**, "SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks," en, **may** 2017, arXiv:1708.04485 [cs]. DOI: 10.48550/arXiv.1708.04485. **url**: http://arxiv.org/abs/1708.04485.

[22] J. Choi **andothers**, "Enabling Fine-Grained Spatial Multitasking on Systolic-Array NPUs Using Dataflow Mirroring," en, *IEEE Transactions on Computers*, **jourvol** 72, **number** 12, **pages** 3383–3398, **december** 2023. DOI: 10.1109/TC.2023.3299030. **url**: https://ieeexplore.ieee.org/document/10198513/.

[23] J.-S. Park **andothers**, "9.5 A 6K-MAC Feature-Map-Sparsity-Aware Neural Processing Unit in 5nm Flagship Mobile SoC," en, **in***2021 IEEE International Solid- State Circuits Conference (ISSCC)* San Francisco, CA, USA: IEEE, **february** 2021, **pages** 152–154, ISBN: 978-1-7281-9549-0. DOI: 10.1109/ISSCC42613.2021.9365928. **url**: https://ieeexplore.ieee.org/document/9365928/.

[24] S. Kim, S. Cho, E. Park **and** S. Yoo, "FPGA Prototyping of Systolic Array-based Accelerator for Low-Precision Inference of Deep Neural Networks," en, **in***2021 IEEE International Workshop on Rapid System Prototyping (RSP)* Paris, France: IEEE, **october** 2021, **pages** 1–7, ISBN: 978-1-6654-6956-2. DOI: 10.1109/RSP53691.2021.9806200. **url**: https://ieeexplore.ieee.org/document/9806200/.

[25] F. Al-Ali, T. D. Gamage, H. W. Nanayakkara, F. Mehdipour **and** S. K. Ray, "Novel casestudy and benchmarking of AlexNet for edge AI: From CPU and GPU to FPGA," **in***2020 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)* IEEE, 2020, **pages** 1–4.

[26] NVIDIA, *Matrix Multiplication Background User's Guide*, en. **urlseen** 18 **october** 2025. **url**: https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html.

[27] E. Qin **andothers**, "SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training," en, **in***2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)* San Diego, CA, USA: IEEE, **february** 2020, **pages** 58–70, ISBN: 978-1-7281-6149-5. DOI: 10.1109/HPCA47549.2020.00015. **url**: https://ieeexplore.ieee.org/document/9065523/.